

PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
Facultad de Ingeniería
IIC2613 Inteligencia Artificial
2026-1

Tarea 4
ASP

Fecha de entrega: Lunes 15 de Junio a las 23:59 hrs

Aspectos generales

Formato y plazo de entrega

El formato de entrega son los archivos especificados a lo largo del enunciado en sus respectivos directorios (`DCCiudad Automatizada` y `DCContainerMarshalling`). El lugar de entrega es en el repositorio de la tarea, en la branch por defecto, hasta el lunes 15 de junio a las 23:59 hrs. Para crear tu repositorio, debes entrar en el enlace del anuncio de la tarea en Canvas. Por último, recuerda que los cupones de atraso son días **reales** (hábiles o no) extra.

Integridad Académica

Este curso se adhiere al Código de Honor establecido por la universidad, el cual tienes el deber de conocer como estudiante. Todo el trabajo hecho en esta tarea debe ser **totalmente individual**. La idea es que te des el tiempo de aprender estos conceptos fundamentales, tanto para el curso, como para tu formación profesional. Las dudas se deben hacer exclusivamente al cuerpo docente a través de las *issues* en GitHub.

Por otra parte, sabemos que estás utilizando material hecho por otras personas, por lo que es importante reconocerlo de la forma apropiada. Todo lo que obtengas de internet debes citarlo de forma correcta (ya sea en formato APA, ICONTEC o IEEE). Cualquier falta a la ética y/o a la integridad académica será sancionada con la reprobación del curso y los antecedentes serán entregados a la Dirección de Pregrado.

El uso de agentes inteligentes—como ChatGPT, Gemini, Copilot u otros—está permitido en las tareas siempre y cuando se indique, al inicio de la respuesta, que si se utilizaron, que se especifique de manera clara su uso (al menos el o los prompts y respuestas) y que las respuestas de estos sean modificadas sustancialmente.

Comentarios adicionales

Es fundamental que pongan énfasis en las justificaciones de sus respuestas, cuidando la redacción, ortografía; manteniendo el código ordenado y comentado. Aquellas respuestas que solo presenten resultados o código (sin contexto ni comentarios) no serán consideradas, mientras que tareas desordenadas pueden ser objeto de descuentos.

1. DCCiudad Automatizada (2 puntos)

Mientras revisabas distintos documentales durante tu semana de receso, te encontraste con *The Choice is Ours*, donde Jacque Fresco plantea que la gestión y escasez de recursos no son problemas políticos, sino que son problemas técnicos que se resuelven mediante la aplicación de la ciencia y la tecnología. En este contexto, propone una sociedad altamente automatizada, donde sistemas inteligentes gestionan de manera eficiente los recursos y procesos necesarios para el funcionamiento de las ciudades.

El *Proyecto Venus* es una organización sin fines de lucro dedicada al diseño de hábitats humanos que buscan, de manera simultánea, mejorar el bienestar de las personas y mantenerse dentro de la capacidad de carga de la Tierra y sus ecosistemas locales. Para ello, explora nuevos materiales, fuentes de energía, métodos de construcción y un nuevo paradigma de gestión de recursos orientado hacia una cultura más humana y sostenible. Sus orígenes se remontan a décadas atrás, representando una continuidad de más de 90 años de trabajo como una de las respuestas más tempranas a la metacrisis global.

Ve la **Parte 3** del documental disponible aquí.

Instrucción de entrega: Se espera que respondas las preguntas realizando una reflexión que utilice los conceptos vistos en el curso. Las respuestas deberán ir en un archivo PDF llamado `respuestas.pdf` dentro del directorio `DCCiudad Automatizada`.

Inspirado/a por estas ideas, decides imaginar cómo podría automatizarse un área relacionada con el desarrollo de la ciudad utilizando la representación del conocimiento y el razonamiento lógico.

Identifica **dos áreas reales** que puedan automatizarse y modelarse mediante técnicas de razonamiento deductivo y representación del conocimiento.

Por ejemplo: El riego automático de plantas en la ciudad podría utilizarse como base para definir objetos, relaciones y reglas lógicas dentro de un sistema automatizado **(no puede ser utilizado)**. A continuación, deberás proponer tus propios ejemplos.

Para cada área, desarrolla:

1. **Definición del problema** — Define el área específica que buscas automatizar, en la cual existe un claro potencial de resolverlo mediante una aplicación de programación en lógica. Explica qué se quiere resolver y por qué sería relevante automatizarlo. **(0.2 puntos)**

2. **Representación del conocimiento** — Explica qué constantes (objetos del mundo real) y predicados (relaciones entre los objetos) serían necesarios para representarlo en una base de conocimiento (KB). **(0.2 puntos)**

3. **Reglas del sistema** — Describe, a un nivel general, de qué tipo serían las reglas utilizadas en el sistema. Explica cómo esas reglas transforman la información captada por sensores en afirmaciones lógicas, y qué es lo que deduce el sistema; finalmente, qué es lo que se debe optimizar en términos de los predicados del sistema. **(0.2 puntos)**

4. **¿Es un problema combinatorial?** — Argumenta si la búsqueda de una solución óptima en tu aplicación propuesta constituye un problema combinatorial relativamente difícil, como los citados en clases. Acá no se espera una demostración sino que solo una discusión intuitiva. **(0.2 puntos)**

5. **¿Requiere otras técnicas o disciplinas?** — Explica por qué el razonamiento deductivo podría no ser suficiente por sí solo. Argumenta si se requieren otras técnicas de inteligencia artificial (como

puede ser visión por computador) o la colaboración con otras áreas de la ingeniería (eléctrica, mecánica, civil, etc.), u otras áreas del conocimiento que no son tan 'técnicas' como la ingeniería (como la psicología, sociología, filosofía, estudio de las artes y letras) **(0.2 puntos)**

Toda respuesta debe estar completamente justificada y debe tener relación con el video propuesto.

2. DCContainerMarshalling (4 puntos)

2.1. Introducción: Un problema en el puerto



Mientras avanzabas en el desarrollo de herramientas basadas en ASP para resolver problemas de planificación y razonamiento, recibiste una invitación inesperada para participar en un proyecto logístico en uno de los principales puertos del país. Durante los últimos años, el aumento del comercio internacional ha provocado una enorme congestión en los patios de contenedores. Los operadores deben reorganizar miles de containers diariamente para permitir su retiro en el orden correcto. Sin embargo, este proceso sigue realizándose de manera altamente manual y muchas veces genera movimientos innecesarios, pérdidas de tiempo y bloqueos operacionales. Uno de los principales problemas ocurre cuando un contenedor que debe salir primero queda atrapado debajo de otros containers con menor prioridad. Para resolverlo, las grúas deben mover temporalmente contenedores entre distintas pilas, intentando mantener el patio ordenado y evitando exceder la altura máxima permitida. Durante una reunión técnica, descubres que este problema puede modelarse naturalmente mediante *Answer Set Programming* (ASP). En particular, el problema consiste en reorganizar containers distribuidos en distintas pilas utilizando movimientos válidos, con el objetivo de alcanzar una configuración ordenada. Inspirado por las aplicaciones vistas en clases, decides construir un sistema inteligente capaz de modelar y resolver automáticamente este problema usando Clingo.

2.2. El problema de Container Marshalling

En un patio de containers existen múltiples pilas verticales. Cada contenedor posee una prioridad representada por un número entero positivo. Mientras menor sea el número, antes debe retirarse el container.

El objetivo del problema es reorganizar las pilas de forma que, en cada pila, los containers estén ordenados correctamente según su prioridad.

Se considerará que una pila está ordenada si:

- Al observarla desde arriba hacia abajo, las prioridades aparecen en orden ascendente. Equivalentemente, si un container está sobre otro, entonces su prioridad debe ser menor o igual.

Por ejemplo:

- La secuencia `1 sobre 4 sobre 7` está ordenada.
- La secuencia `7 sobre 3` no está ordenada.

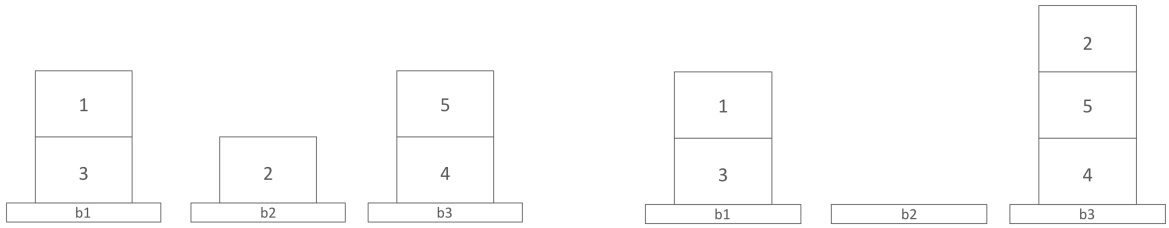


Figura 1: A la izquierda: estado inicial ejemplo con tres bases de pila y 5 containers. Los containers que se encuentran libres son 1, 2 y 5. A la derecha: el resultado de ejecutar `move(2,5,0)` en el estado inicial.

2.3. Representación de una instancia del problema

Representaremos el estado del patio a través de 3 predicados. El primero:

```
on(C1,C2,T).
```

indica que el container `C1` se encuentra sobre el container o base `C2` en el tiempo `T`. Es segundo predicado se utiliza representar las "bases de pilas", que intuitivamente representan aquel lugar sobre el cual se pueden apilar containers. Así,

```
stack_base(B).
```

establece que existe una base de pila `B`.

Para representar los containers que existen, utilizamos el predicado `container`.

```
container(C).
```

que establece que `C` es un container. En cada instancia de prueba que utilizaremos (veremos los detalles más adelante), `C` corresponde a un número.

Las siguientes líneas ASP, por tanto, representan al estado mostrado en la figura 1, donde existen 3 bases en donde se pueden apilar containes, el container `1` está sobre el `3`, mientras que el `3` se encuentra sobre la pila `b1`.

```
stack_base(b1).
stack_base(b2).
stack_base(b3).

container(1).
container(2).
container(3).
container(4).
container(5).

on(1,3,0).
on(3,b1,0).
on(2,b2,0).
on(4,b3,0).
on(5,4,0).
```

2.4. Representación de acciones y cambio

Representamos la acción de mover un container `C1` sobre la base de pila o container `C2` en el tiempo `T` de la siguiente manera:

```
move(C1,C2,T).
```

Por ejemplo `move(1,5,0)` indica que el container `1` se mueve sobre el container `5` en el tiempo `0`. Reflexiona sobre las siguientes

1. ¿Cuál es el efecto positivo de la acción `move(C1,C2,T)`? Anótalo en palabras.
2. ¿Cuál es el efecto negativo de la acción `move(C1,C2,T)`? Anótalo en palabras.
3. ¿Cuál es la precondición de esta acción? Anótala en palabras.

2.5. Restricción de altura

Cada pila posee una altura máxima permitida. No se puede ejecutar un movimiento que provoque que una pila exceda dicha altura. En la definición de cada instancia, el predicado `hmax` define la altura máxima de cada pila. Por ejemplo:

```
hmax(5).
```

establece que la altura máxima de cada pila, en todo momento es 5, es decir, que pueden haber a lo más 5 bloques apilados, uno sobre el otro, sobre cada base de pila.

2.6. Objetivo

El objetivo es encontrar una secuencia válida de movimientos que reorganice correctamente todas las pilas de containers. Una configuración se considera desordenada si existe un container con prioridad mayor sobre otro de prioridad menor.

2.7. Instancias de prueba

Junto a la tarea se entregará una carpeta `tests/` con instancias de prueba organizadas según cada desafío. La estructura será la siguiente:

```
tests/
    D1/
    D2/
    D3/
    D4/
```

donde `DX` corresponde al número del desafío asociado.

Cada archivo de prueba contendrá una instancia particular del problema y podrá incluir un predicado de la forma:

```
#const bound = N.
```

Este predicado define el horizonte temporal máximo considerado para resolver la instancia. Es decir, representa la cantidad máxima de pasos disponibles para ejecutar movimientos. Sin embargo, una solución válida puede resolver el problema utilizando menos pasos.

Los tests entregados tienen únicamente un propósito referencial y de apoyo al debugging. Las instancias utilizadas durante la evaluación no necesariamente serán las mismas. Por esta razón, se recomienda fuertemente que cada estudiante cree sus propios casos de prueba para validar correctamente el comportamiento de su solución.

2.8. Estructura de los desafíos

La tarea se encuentra dividida en cuatro desafíos progresivos. La idea es construir el modelo paso a paso, verificando constantemente que el comportamiento del sistema sea correcto antes de agregar nuevas restricciones. Se recomienda probar cada desafío utilizando el visualizador entregado junto a la tarea.

Importante: durante los primeros desafíos es completamente normal que el modelo aún presente comportamientos inesperados o configuraciones inválidas. Esto ocurre porque muchas de las restricciones importantes del problema todavía no han sido incorporadas. Además, en las primeras partes pueden existir instantes de tiempo donde no se ejecute ninguna acción. Recién en el desafío 4 se solicitará optimizar las soluciones minimizando la cantidad de movimientos y el tiempo total requerido.

2.8.1. Desafío 1 — Dinámica básica del mundo (0.5 puntos)

En este primer desafío deberás modelar la representación básica del mundo y la evolución de estados en el tiempo. En particular, deberás correctamente la dinámica del predicado `on` respecto de la acción `move`. Esto significa que, dada cualquier secuencia de movimientos legal que se incluya como parte de tu programa, por ejemplo,

```
move(1,5,0).
move(3,1,1).
```

Debería ser verdadero en tiempo 1 que el container 1 está sobre el container 5, y en tiempo 2 que el container 3 está sobre 1. Así mismo, de usar la instancia de ejemplo en el enunciado, en tiempo 1 **no** debiera ser verdad que el container 5 está sobre el container 4, y en tiempo 2 **no** debiera ser verdad que container 1 está sobre 3. Finalmente, tanto en tiempo 1 como en 2 los bloques que no han sido movidos permanecen sobre los mismos bloques o bases de pila.

Algo importante a notar en este punto es que no has implementado las restricciones aún, esto significa que podrías incluir acciones ilegales sin quererlo. Al hacer tus propias pruebas, utiliza solo acciones legales. **No es necesario que tu programa, en este punto, maneje correctamente acciones que no son posibles**.

Al utilizar el visualizador, los movimientos realizados deberían verse reflejados correctamente en la simulación.

Qué entregar

En el subdirectorio `DCContainerMarshalling/`:

- Un archivo `parte1.lp`.

2.8.2. Desafío 2 — Precondiciones de la acción move (0.5 puntos)

Una vez implementada correctamente la dinámica del predicado on, te enfocarás en las precondiciones de la acción move. El desafío mayor acá es ser capaz es que idees una manera de representar cuáles son los bloques que están *libres*, es decir, están al tope de alguna pila. Te dejaremos a ti diseñar un predicado que permita realizar esto.

A partir de esto, deberás introducir restricciones que impidan movimientos inválidos. En particular:

- Un container solo puede moverse si se encuentra libre.
- Un container no puede posicionarse sobre otro container que no esté en el tope de una pila.

Por ejemplo, si existe un container sobre el container `1`, en tiempo 0, entonces el siguiente movimiento debe ser inválido:

```
move(1,9,0).
```

y aquello debiera reflejarse en que Clingo determinara que ese programa no tiene conjuntos respuesta.

Habrás superado este desafío cuando tu programa permita secuencias válidas, calculando correctamente los cambios en el mundo a través del predicado `on`, y, por otro lado, generando una respuesta `UNSATISFIABLE` cuando se declaran movimientos inválidos.

Qué entregar

En el subdirectorio `DCContainerMarshalling/`:

- Un archivo `parte2.lp`.

2.8.3. Desafío 3 — Encontrando soluciones (1.5 puntos)

Este desafío tiene tres objetivos.

El primer objetivo consiste en agregar una restricción que exprese que cada pila posee una altura máxima permitida determinada por el predicado `hmax`. Por lo tanto, no es válido ejecutar movimientos que provoquen que una columna exceda dicha altura.

Una recomendación para resolver este problema es:

- Representar una relación *sobre* que permita inferir si un container está en la pila que nace desde una base de pila.
- Definir un predicado que retorne la altura de la pila sobre una cierta base de pila.
- Escribir una restricción que prohiba que alguna pila tenga altura 6 o más.

El segundo objetivo consiste en definir un predicado `ordenado(T)` que se deduzca del programa, cuando todas las pilas están en un orden legal en `T`. Puede convenir definir este predicado en términos de otro; por ejemplo, un predicado `desordenado(T)`.

El tercer objetivo es agregar una restricción que permita a Clingo encontrar soluciones al problema; es decir, encontrar una secuencia válida de movimientos que deje todas las pilas ordenadas respetando la restricción de altura.

Qué entregar

En el subdirectorio `DCContainerMarshalling/`:

- Un archivo `parte3.lp`.

2.8.4. Desafío 4 — Llegada dinámica de containers (1.5 puntos)

En este último desafío el problema se vuelve más dinámico. Ahora no todos los containers se encuentran inicialmente en el patio, sino que algunos pueden llegar a un área de descarga adyacente a las pilas, en distintos instantes de tiempo.

La llegada de nuevos containers se representa, en la definición de la instancia del problema, mediante:

```
arrive(C,T).
```

lo que significa que en tiempo `T` el container `C` "llega" al área adyacente a las pilas. Lo importante a entender acá es que `C` no está sobre otro container en `T`, sino que en un área de espera.

Deberás modelar la dinámica del predicado:

```
available(C,T).
```

el cual representa que el container `C` ya llegó al patio y se encuentra disponible o esperando para ser recogido e ingresado a las columnas. Este predicado, cambia en el tiempo debido a la acción

```
drop(C,B,T).
```

la cual representa que el container `C` deja el estado `available` y es colocado sobre la pila cuya base es `B` en el tiempo `T`. Asimismo, `drop` tiene también un efecto sobre el predicado `on`, el cual también deberás modelar.

Los containers disponibles para ingresar al patio deben manejarse como una cola FIFO (*First In First Out*).

Esto significa que:

- El primer container que llega debe ser el primero en ingresar al patio.
- No se puede ingresar un container posterior mientras exista uno anterior esperando.

Puedes suponer que no habrán dos hechos `arrive` definidos en la instancia para el mismo tiempo `T`.

Finalmente, debes considerar que la acción `drop` no se puede ejecutar en paralelo con una acción `move`.

En resumen, para este desafío debes implementar:

- El predicado `available/2`.
- La acción `drop/3`, con sus efectos sobre `available/2` y `on/3`.
- Restricciones FIFO.
- No-paralelismo entre `drop` y `move`.

Finalmente, el sistema deberá ser capaz de encontrar una configuración ordenada:

- respetando las restricciones de altura,
- respetando el orden FIFO de llegada,
- minimizando la cantidad de movimientos,
- y alcanzando la solución en el menor tiempo posible.

Este desafío corresponde a la versión completa del problema de *Container Marshalling*.

Qué entregar

En el subdirectorio `DCContainerMarshalling/`:

- Un archivo `parte4.lp`.

2.9. Visualizador

Junto a la tarea se entrega un visualizador desarrollado en Python para inspeccionar gráficamente las soluciones generadas por Clingo.

El visualizador puede ejecutarse mediante el comando:

```
python visualizer.py [solucion.lp] [test.lp] [nombre_visualizacion.html]
```

Por ejemplo:

```
python visualizer.py pauta_d3.lp tests/D3/d3_test1.lp visualizacion.html
```

Al ejecutarse, el visualizador generará un archivo HTML llamado `visualizacion.html` (o el nombre indicado por el usuario).

Este archivo debe abrirse en un navegador web para poder visualizar gráficamente cómo la solución es ejecutada paso a paso.

El visualizador permite:

- avanzar paso a paso,
- observar movimientos,
- visualizar drops,
- y verificar visualmente el orden de las pilas.

Se recomienda utilizar esta herramienta constantemente para debugging y validación de soluciones.

Qué entregar

En un subdirectorio `DCContainerMarshalling/`:

- `parte1.lp`
- `parte2.lp`
- `parte3.lp`
- `parte4.lp`
- `respuesta.pdf`

El PDF debe incluir:

- explicación de las decisiones de modelamiento,
- pruebas realizadas,
- dificultades encontradas,
- y reflexión sobre el uso de ASP para resolver problemas de planificación.